

SwarmOps Cloud

API Reference — Storefront and Commerce Administration

September 2026

Abstract

This reference documents the two HTTP interfaces that SwarmOps Cloud adds to SwarmOps Core: the storefront API at `/api/store/v1`, used by customers' browsers, and the commerce administration API at `/api/v1/commerce`, used by the administration console. For every endpoint it gives the method and path, the protection that applies, the request body and the response. It also documents the conventions all endpoints share: JSON encoding, authentication and CSRF, rate limiting, idempotency, pagination and the error model. The operations API that SwarmOps already provided is out of scope.

Contents

1	Conventions	2
1.1	Encoding	2
1.2	Authentication	2
1.3	Rate limiting	2
1.4	Idempotency	2
1.5	Pagination and filters	3
1.6	Errors	3
2	Resource representations	3
2.1	User	3
2.2	Session	4
2.3	Plan	4
2.4	Wallet and Transaction	4
2.5	Order and OrderItem	4
2.6	Project	4
2.7	Invoice and InvoiceLine	5
2.8	Ticket and TicketMessage	5
3	Storefront API	5
3.1	Summary	5
3.2	Plans	6
3.3	Registration and sign-in	6
3.4	Wallet	6
3.5	Orders	6
3.6	Projects and invoices	7
3.7	Support tickets	7
4	Commerce administration API	7
4.1	Summary	7
4.2	Overview and reports	8
4.3	Customers and wallets	9
4.4	Plans	9
4.5	Orders	9
4.6	Projects, invoices and billing	10
4.7	Tickets	10
5	Example session	10

List of Figures

List of Tables

1	Authentication of the two interfaces	2
2	Protection levels	2
3	Error model	3
4	Storefront endpoints	5
5	Order fields	6
6	Commerce administration endpoints	7

1 Conventions

1.1 Encoding

Requests and responses are JSON encoded in UTF-8, with **Content-Type: application/json**. Field names use lower camel case. Timestamps are RFC 3339 strings in UTC, for example 2026-09-15T17:12:24.010856Z. Money is an integer number of **rials**; the interfaces show it in tomans (rials $\div$ 10). Identifiers are integers, except command identifiers, which are strings.

1.2 Authentication

Table 1: Authentication of the two interfaces

API	Session	Changes also require
Storefront (/api/store/v1)	Cookie swarmops_store_session , set by sign-in: HttpOnly, SameSite=Strict, Path=/, Secure when secure cookies are configured, and expiring with the session (12 hours)	Header X-CSRF-Token equal to the csrfToken returned by sign-in or by GET /auth/me
Commerce administration (/api/v1/commerce)	The console's operator session	Header X-CSRF-Token , and this core must be the active core

The session token itself never appears in a response body; it travels only in the cookie.

Throughout this document the **Protection** column uses these abbreviations:

Table 2: Protection levels

Abbreviation	Meaning
Public	No session needed.
Customer	A valid storefront session.
Customer + CSRF	A valid storefront session and the CSRF header.
Operator	A valid operator session.
Operator + CSRF + active	A valid operator session, the CSRF header, and this core being the active core. A standby core answers 409.

1.3 Rate limiting

After eight failed attempts from the same source within fifteen minutes, registration and sign-in answer 429 **Too Many Requests**. Registration counts attempts per client address. Sign-in counts them per client address and email address. The client address honours **X-Forwarded-For** only from configured trusted proxies.

1.4 Idempotency

POST /wallet/topups and POST /customers/{id}/adjustments require an **Idempotency-Key** header. Repeating a request with the same key for the same customer returns the original

transaction and moves no money. Confirmation and refunds use keys derived from the order (`order-charge-<id>`, `cloud-order-<id>`, `order-refund-<id>`), so they are idempotent without a header.

1.5 Pagination and filters

List endpoints accept `limit`, from 1 to 500; a missing or out-of-range value means 100. Some lists also accept a `status` filter. Results are ordered newest first unless stated otherwise.

1.6 Errors

Every error has a JSON body with an `error` message written for the person using the page. A validation error also names the `field`.

Table 3: Error model

Status	Meaning	Body
400	The body is not valid JSON or has unknown fields.	<code>{"error"}</code>
401	No session, an expired session, or wrong credentials.	<code>{"error"}</code>
403	Missing or wrong CSRF token, or the account may not do this.	<code>{"error"}</code>
404	The resource does not exist or belongs to another customer.	<code>{"error"}</code>
409	A conflict: the name is taken, the order is no longer pending, the balance is insufficient, or this core is standby.	<code>{"error"}</code> ; insufficient reserve also returns <code>balanceRial</code> and <code>requiredRial</code>
422	Validation failed.	<code>{"error", "field"}</code>
429	Too many sign-in or registration attempts.	<code>{"error"}</code>
500	Unexpected failure; the cause is logged with the request id.	<code>{"error", "requestId"}</code>
503	The commerce service or a dependency (command storage, audit ledger) is unavailable.	<code>{"error"}</code>

Example:

HTTP/1.1 422 Unprocessable Entity

```
{"error": "use between 10 and 72 characters", "field": "password"}
```

2 Resource representations

2.1 User

```
{"id": 1, "email": "sara.demo@example.com", "fullName": "Sara Ahmadi",  
  "role": "customer", "status": "active",  
  "createdAt": "2026-09-15T16:43:31Z", "lastLoginAt": "2026-09-15T17:33:40Z"}
```

role is customer or admin; status is active or suspended.

2.2 Session

```
{"csrfToken": "...", "expiresAt": "2026-09-16T04:43:31Z", "user": { "...": "User" }}
```

2.3 Plan

```
{"id": 1, "code": "starter", "name": "Starter", "category": "app",  
  "description": "A small API, a bot, or a personal site.",  
  "descriptionFa": " API",  
  "cpuMillicores": 250, "memoryMiB": 256, "diskGiB": 5,  
  "hourlyPriceRial": 4000, "monthlyPriceRial": 2700000,  
  "features": ["Runs on Docker Swarm and is replaced if its health check fails", "..."],  
  "featuresFa": [ " ", "..."],  
  "active": true, "sortOrder": 10, "updatedAt": "..."}  
descriptionFa and featuresFa are the Persian text shown on the Persian storefront; either may be empty, in which case the storefront shows the English text.
```

2.4 Wallet and Transaction

```
{"userId": 1, "balanceRial": 4996000, "updatedAt": "..."}  
  
{"id": 5, "userId": 1, "kind": "order_charge", "amountRial": -4000,  
  "balanceAfterRial": 4992000, "referenceType": "order", "referenceId": 3,  
  "description": "First hour of sara-api", "createdAt": "..."}  
kind is one of topup, order_charge, usage, refund or adjustment. amountRial is negative for money leaving the wallet.
```

2.5 Order and OrderItem

```
{"id": 3, "userId": 1, "status": "active", "upfrontRial": 4000,  
  "customerNote": "This image answers /healthz on 8080.",  
  "customerEmail": "sara.demo@example.com", "reviewReason": "",  
  "placedAt": "...", "reviewedAt": "...", "updatedAt": "...",  
  "items": [{"id": 3, "planCode": "starter", "planName": "Starter",  
    "appName": "sara-api", "image": "mendhak/http-https-echo:41", "port": 8080,  
    "hourlyPriceRial": 4000, "monthlyPriceRial": 2700000}]}  
status is one of pending, rejected, cancelled, provisioning, active or failed.  
customerEmail appears only in administration responses.
```

2.6 Project

```
{"id": 3, "userId": 1, "orderItemId": 3, "appName": "sara-api",  
  "planCode": "starter", "planName": "Starter", "serverId": "server-...",  
  "commandId": "cmd-7dd041e2...", "status": "active",  
  "hourlyPriceRial": 4000, "monthlyPriceRial": 2700000,  
  "usageThisMonthRial": 4000,  
  "createdAt": "...", "activatedAt": "...", "suspendedAt": null}  
status is one of provisioning, active, suspended, failed or deleted.
```

2.7 Invoice and InvoiceLine

```
{ "id": 1, "userId": 1, "number": "INV-202609-000001", "status": "paid",  
  "periodStart": "2026-09-01T00:00:00Z", "periodEnd": "2026-09-30T00:00:00Z",  
  "subtotalRial": 10910, "taxRateBp": 1000, "taxRial": 1090, "totalRial": 12000,  
  "issuedAt": "...", "createdAt": "...",  
  "lines": [{ "id": 1, "projectId": 3, "description": "sara-api on the Starter plan, September",  
    "quantityHours": 3, "amountRial": 12000 } ] }
```

`lines` is included only when a single invoice is read. The tax contained in a total T at a rate of r basis points is $T \cdot r / (10000 + r)$, and `subtotalRial` = `totalRial` - `taxRial`. The numbers above illustrate the formula.

2.8 Ticket and TicketMessage

```
{ "id": 1, "userId": 1, "subject": "How do I reach sara-api from the internet?",  
  "priority": "normal", "status": "answered", "projectId": 3,  
  "customerEmail": "sara.demo@example.com", "createdAt": "...", "updatedAt": "...",  
  "messages": [ { "id": 1, "authorName": "Sara Ahmadi", "authorRole": "customer",  
    "body": "...", "createdAt": "..."} ] }
```

`messages` is included when a single ticket is read.

3 Storefront API

Base path: `/api/store/v1`.

3.1 Summary

Table 4: Storefront endpoints

Method	Path	Protection	Success
GET	<code>/plans</code>	Public	200 <code>Plan[]</code>
POST	<code>/auth/register</code>	Public, rate-limited	201 <code>User</code>
POST	<code>/auth/login</code>	Public, rate-limited	200 <code>Session</code> , sets the cookie
POST	<code>/auth/logout</code>	Customer + CSRF	204, clears the cookie
GET	<code>/auth/me</code>	Customer	200 <code>Session</code>
GET	<code>/wallet</code>	Customer	200 <code>{wallet, transactions}</code>
POST	<code>/wallet/topups</code>	Customer + CSRF, Idempotency-Key	200 <code>Transaction</code>
GET	<code>/orders</code>	Customer	200 <code>Order[]</code>
POST	<code>/orders</code>	Customer + CSRF	201 <code>Order</code>
GET	<code>/orders/{id}</code>	Customer	200 <code>Order</code>
POST	<code>/orders/{id}/cancel</code>	Customer + CSRF	200 <code>Order</code>
GET	<code>/projects</code>	Customer	200 <code>Project[]</code>
GET	<code>/invoices</code>	Customer	200 <code>Invoice[]</code>
GET	<code>/invoices/{id}</code>	Customer	200 <code>Invoice</code> with <code>lines</code>
GET	<code>/tickets</code>	Customer	200 <code>Ticket[]</code>
POST	<code>/tickets</code>	Customer + CSRF	201 <code>Ticket</code>

Method	Path	Protection	Success
GET	/tickets/{id}	Customer	200 Ticket with messages
POST	/tickets/{id}/replies	Customer + CSRF	200 Ticket
POST	/tickets/{id}/close	Customer + CSRF	200 Ticket

3.2 Plans

GET /plans returns the plans currently offered, in display order.

3.3 Registration and sign-in

POST /auth/register creates a customer account and its wallet.

```
{"email": "sara.demo@example.com", "fullName": "Sara Ahmadi", "password": "at least ten char"}
```

The email address must be valid and unused, the full name is required, and the password must be 10 to 72 characters. A used email address gives 409. Registration does not sign in; the storefront calls sign-in next.

POST /auth/login takes {"email", "password"}. It answers with a **Session** and sets the session cookie. Wrong credentials give 401 Invalid email or password, and a suspended account gives 403.

POST /auth/logout revokes the session.

GET /auth/me returns the current session, including a fresh copy of its CSRF token. The storefront calls it when a page loads, and again once when a change is refused because the page's token is out of date.

3.4 Wallet

GET /wallet returns {"wallet": **Wallet**, "transactions": **Transaction**[]}, the ledger newest first. It accepts limit.

POST /wallet/topups simulates a gateway confirmation and credits the wallet. It requires the Idempotency-Key header.

```
{"amountRial": 5000000}
```

The amount must be between 100,000 and 2,000,000,000 rials. A successful top-up also resumes suspended projects that the new balance can pay for.

3.5 Orders

POST /orders places an order.

```
{"planCode": "starter", "appName": "sara-api", "image": "mendhak/http-https-echo:41", "port": 8080, "note": "This image answers /healthz on 8080."}
```

Table 5: Order fields

Field	Rule
planCode	An offered plan.

Field	Rule
appName	A lowercase letter followed by up to 40 lowercase letters, digits or hyphens; not used by a project, an order in progress, or an application. A taken name gives 409.
image	No spaces; either a digest (@sha256:...) or a tag other than latest.
port	1–65535; the port where the application answers GET /healthz.
note	Optional, under 500 characters.

GET /orders and GET /orders/{id} read the customer's own orders. Another customer's order answers 404.

POST /orders/{id}/cancel cancels a pending order. An order that is not pending gives 409.

3.6 Projects and invoices

GET /projects lists the customer's projects with this month's charges. GET /invoices lists their invoices, and GET /invoices/{id} returns one invoice with its lines.

3.7 Support tickets

POST /tickets opens a ticket:

```
{"subject": "How do I reach sara-api from the internet?",
  "body": "The project page says sara-api is active, ...", "priority": "normal", "projectId": 3}
```

- The subject must be 3–160 characters and the body under 5,000 characters.
- priority is low, normal or high.
- projectId is optional; 0 means no project, and any other value must be one of the customer's own projects.

GET /tickets accepts status (open, answered or closed) and limit. POST /tickets/{id}/replies takes {"body"}. POST /tickets/{id}/close closes the ticket.

4 Commerce administration API

Base path: /api/v1/commerce. If the commerce service is not enabled on the controller, every endpoint answers 503.

4.1 Summary

Table 6: Commerce administration endpoints

Method	Path	Protection	Success
GET	/overview	Operator	200 Overview
GET	/customers	Operator	200 CustomerAccount []
GET	/customers/{id}/wallet	Operator	200 {wallet, transactions, ledgerConsistent}

Method	Path	Protection	Success
POST	/customers/{id}/status	Operator + CSRF + active	204
POST	/customers/{id}/adjustments	Operator + CSRF + active,	200 Transaction
GET	/plans	Idempotency-Key Operator	200 Plan[], including withdrawn plans
PUT	/plans/{code}	Operator + CSRF + active	200 Plan
GET	/orders	Operator	200 Order[]
POST	/orders/{id}/confirm	Operator + CSRF + active	200 {order, project, command}
POST	/orders/{id}/reject	Operator + CSRF + active	200 Order
GET	/projects	Operator	200 Project[]
GET	/invoices	Operator	200 Invoice[]
GET	/invoices/{id}	Operator	200 Invoice with lines
POST	/invoices/issue	Operator + CSRF + active	200 {issued}
POST	/billing/run	Operator + CSRF + active	200 BillingResult
GET	/revenue	Operator	200 RevenueMonth[]
GET	/tickets	Operator	200 Ticket[]
GET	/tickets/{id}	Operator	200 Ticket with messages
POST	/tickets/{id}/replies	Operator + CSRF + active	200 Ticket
POST	/tickets/{id}/close	Operator + CSRF + active	200 Ticket

Every change is recorded in the audit ledger under the operator's name.

4.2 Overview and reports

GET /overview returns:

```
{"customers": 1, "pendingOrders": 0, "activeProjects": 1, "suspendedProjects": 0,
  "openTickets": 0, "revenueThisMonthRial": 4000, "walletFloatRial": 4996000}
```

revenueThisMonthRial is charges minus refunds since the first day of the month. walletFloatRial is the total balance customers hold.

GET /revenue?months=12 returns one entry per month, newest first: {"month": "2026-09", "chargedRial", "refundedRial", "toppedUpRial", "payingCustomers"}. months ranges from 1 to 60; a missing or out-of-range value means 12.

4.3 Customers and wallets

GET /customers lists CustomerAccount entries:

```
{"userId": 1, "email": "sara.demo@example.com", "fullName": "Sara Ahmadi", "status": "active",  
  "balanceRial": 4996000, "liveProjects": 1, "pendingOrders": 0, "createdAt": "..."}"
```

GET /customers/{id}/wallet returns the wallet, its ledger, and ledgerConsistent, which is true when the balance equals the sum of the ledger.

POST /customers/{id}/status takes {"status": "active" | "suspended"}. Suspending revokes every session of the customer.

POST /customers/{id}/adjustments records a correction:

```
{"amountRial": -50000, "reason": "Refund of a duplicate top-up"}
```

The amount must not be zero, the reason is required, and the header Idempotency-Key is required. An adjustment that would make the balance negative gives 409.

4.4 Plans

PUT /plans/{code} creates or updates a plan:

```
{"code": "starter", "name": "Starter", "description": "A small API, a bot, or a personal site",  
  "descriptionFa": "API",  
  "cpuMillicores": 250, "memoryMiB": 256, "diskGiB": 5,  
  "hourlyPriceRial": 4000, "monthlyPriceRial": 2700000,  
  "features": [...], "featuresFa": [...], "active": true, "sortOrder": 10}
```

- code: 2–32 lowercase letters, digits or hyphens.
- description and descriptionFa: each under 255 characters.
- featuresFa: the same rules as features; saving replaces both lists.
- cpuMillicores: 100–64,000.
- memoryMiB: 128–262,144.
- hourlyPriceRial: greater than 0; monthlyPriceRial must be at least the hourly price.
- features: at most 12, each under 160 characters.

Setting active to false withdraws the plan from the storefront. Existing orders keep their prices.

4.5 Orders

GET /orders?status=pending lists orders across customers, filtered by status.

POST /orders/{id}/confirm confirms a pending order on a server:

```
{"serverId": "server-6ebbbe6b6e4790724443ea3d"}
```

Checks, in order:

1. Mutations are enabled; otherwise 403.
2. Command storage is writable; otherwise 503.
3. The audit ledger is writable; otherwise 503.
4. The server is saved; otherwise 422 with field: serverId.
5. The server is connected to a Swarm manager; otherwise 409.
6. The application passes planning on that server; otherwise 422 with the planner's message.
7. The wallet holds 24 hours of the price; otherwise 409 with balanceRial and requiredRial.

On success the first hour is charged, the project created and the deployment queued, in one transaction. The response is:

```

{"order": { "...": "Order, status provisioning" },
 "project": { "...": "Project, status provisioning" },
 "command": {"id": "cmd-7dd041e2...", "action": "application.deploy",
             "state": "queued", "target": "application/sara-api", "...": "..."}}

```

The order and project become **active** or **failed** when the command finishes. A failed deployment is refunded automatically.

POST /orders/{id}/reject takes {"reason"}, which is required and shown to the customer.

4.6 Projects, invoices and billing

GET /projects?status=active lists projects across customers.

POST /invoices/issue takes {"month": "2026-08"} in YYYY-MM form and returns {"issued": n}, the number of invoices created. Issuing a month twice creates none the second time. A malformed month gives 422 Give the month as YYYY-MM.

POST /billing/run runs hourly billing immediately and returns {"chargedHours", "chargedRial", "suspended"}. Hours already charged are not charged again.

4.7 Tickets

GET /tickets?status=open lists tickets across customers. **POST /tickets/{id}/replies** takes {"body"} and marks the ticket answered. **POST /tickets/{id}/close** closes it.

5 Example session

The storefront's calls when a customer registers, tops up and orders:

```

POST /api/store/v1/auth/register      → 201
POST /api/store/v1/auth/login         → 200  Set-Cookie: swarmops_store_session=...
GET  /api/store/v1/wallet             → 200
POST /api/store/v1/wallet/topups      → 200  (X-CSRF-Token, Idempotency-Key)
POST /api/store/v1/orders             → 201

```

The console's calls when an administrator confirms the order:

```

GET  /api/v1/commerce/orders?status=pending → 200
POST /api/v1/commerce/orders/3/confirm      → 200  (X-CSRF-Token)

```

These status codes are those recorded in the browser during the end-to-end run described in the Test Plan and Report.